

Cross-Workload Flow-Size Prediction for AI Data Center Networks

Leonardo Alberro^{*†}, Pedro Casas[†], Matías Richart^{*}, Eduardo Grampin^{*}

^{*}Universidad de la República, Montevideo, Uruguay [†]AIT Austrian Institute of Technology, Vienna, Austria
{lalberro, grampin, mrichart}@fing.edu.uy, pedro.casas@ait.ac.at

Abstract—Modern congestion control mechanisms for data center networks (DCNs) increasingly depend on early knowledge of flow size to differentiate latency-sensitive short flows from throughput-oriented long flows. AI workloads exhibit recurring communication patterns that make flow-size prediction feasible, but existing learning-based approaches typically predict an ongoing flow or rely on workload-specific models. This paper presents a unified framework for *pre-arrival next-flow forecasting*: predicting the size of the next flow before observing any packet belonging to it, using preceding communication history and end-host/workload telemetry. We develop a single predictor across heterogeneous AI workloads by aligning partially overlapping telemetry into a common feature space. We evaluate it from both machine-learning and networking perspectives, combining regression and short/long classification metrics with end-to-end flow completion time (FCT) in a state-of-the-art flow-aware transport protocol. In the primary supervised cross-workload setting, the unified predictor achieves over 98% short/long flow classification accuracy while providing a practical trade-off between predictive performance and the complexity of maintaining workload-specific models. We additionally perform leave-one-workload-out (LOWO) evaluation, showing workload-dependent zero-shot performance and suggesting potential for generalization beyond workload types represented during training, while identifying cases where retraining or adaptation is required. Transport-level results further show how directional prediction errors translate into FCT degradation. To foster transparency and reproducibility, we release the datasets, trained models, implementation, and simulation modules.

Index Terms—Data Center Networks, Transport Protocols, Machine Learning.

I. INTRODUCTION

Data center networks (DCNs) support latency-sensitive and bandwidth-intensive applications at massive scale. In these environments, transport-layer performance is strongly influenced by how congestion control mechanisms react to heterogeneous traffic demands. Although TCP remains the dominant transport protocol, its behavior is known to be suboptimal in DCNs, where shallow buffers, high fan-in communication patterns, and low-latency requirements expose considerable limitations [1]. These directly affect flow completion time (FCT), throughput, and overall resource efficiency.

A central observation in DCN transport design is that not all flows should be treated equally. Short flows are typically latency-sensitive and disproportionately affected by queueing delays, whereas long flows dominate byte volume and primarily determine throughput and buffer occupancy. This asymmetry has motivated several flow-aware congestion control mechanisms [2]–[4], which distinguish short and long

flows and assign them different transport behaviors. Their effectiveness, however, depends on identifying the flow class early enough to influence transport decisions.

This distinction is increasingly important in modern AI data centers, where large-scale distributed training and inference impose strict performance and efficiency requirements. Even small communication inefficiencies can increase job completion time, reduce accelerator utilization, and raise operational cost [5]. At the same time, AI workloads often exhibit repetitive and structured communication behavior [6], [7], creating opportunities to anticipate upcoming transfers.

Obtaining flow-size information sufficiently early remains challenging. Some flow-aware mechanisms assume exact flow size is available before transmission, e.g., through application-provided information [3], an assumption difficult to satisfy in general deployments. Other approaches classify or predict flows after they have started, using transmitted bytes or partial-flow information [8]. Recent work pushes prediction toward flow arrival, considering classification from the first packet [13], prediction of newly arriving flows from historical behavior once their first packet is observed [11], or two-stage approaches combining first-packet classification with prediction from the first n packets [12]. In contrast, our setting forecasts the *next flow before observing any packet belonging to it*, using preceding communication history and available end-host/workload telemetry. We refer to this as *pre-arrival next-flow forecasting*.

Our work builds directly on learning-based approaches that predict flow size from historical communication and end-host telemetry [9], [10]. These studies demonstrate the feasibility of such prediction for structured workloads, but train a separate model for each workload. We follow the same fundamental prediction paradigm, while asking whether it can be extended to a *single cross-workload predictor*. This is operationally relevant: workload-specific solutions require identifying the workload, selecting and maintaining the corresponding model, and managing separate training and lifecycle pipelines.

Cross-workload prediction is complicated by feature heterogeneity. Different AI workloads and frameworks expose different host-, application-, temporal-, and network-level telemetry. Their datasets therefore do not necessarily share the same feature space, preventing straightforward joint training. Restricting all workloads to their common features, on the other hand, discards potentially useful workload-specific information. This motivates our central question: *can pre-arrival*

flow-size prediction be performed through a single model across heterogeneous AI workloads while retaining sufficient predictive quality for flow-aware transport?

We study this question by formulating pre-arrival cross-workload flow-size prediction over heterogeneous feature spaces and investigating feature-alignment strategies that allow a single supervised model to exploit partially overlapping telemetry. Our primary setting considers known workload types represented during training; we additionally evaluate generalization to entirely unseen workload types through leave-one-workload-out experiments. We evaluate the predictors at two levels: first through regression and transport-relevant classification metrics, including directional short-/long errors; and second by integrating their predictions into Fork [3], a flow-aware transport mechanism that assumes oracle flow-size knowledge, and measuring the resulting impact on FCT.

Our results show that unified prediction provides a practical trade-off between predictive performance and cross-workload applicability, while the transport experiments quantify how prediction errors translate into FCT degradation. The specific contributions are:

- **Pre-arrival cross-workload flow-size prediction:** we extend historical-flow and end-host-telemetry-based prediction to a unified setting across heterogeneous AI workloads, forecasting the next flow before observing any packet belonging to it.
- **Heterogeneous feature alignment:** we develop and evaluate strategies that enable a single model to learn from partially overlapping telemetry spaces while retaining workload-specific features.
- **Transport-oriented evaluation:** we bridge predictive quality and transport performance through regression, directional short/long classification errors, and end-to-end integration with Fork.
- **Transport-layer insights:** we quantify how different prediction errors affect FCT, showing that underestimating large flows is substantially more harmful than overestimating short flows and identifying transport-relevant error characteristics for future predictors.
- **Open datasets and reproducibility:** we extend existing public datasets with updated distributed Spark workloads and a GPT fine-tuning workload, providing modern per-flow and end-host telemetry traces together with models and evaluation artifacts released to the community [14].

II. BACKGROUND AND RELATED WORK

Modern distributed training frameworks often reuse persistent TCP connections to avoid connection-setup overhead and support frequent exchanges of small messages. A long-lived connection identified by the standard 5-tuple may therefore carry multiple logically distinct bursts. Following prior work [9], [10], we define a flow as a burst of packets between two hosts separated from other bursts by an inactivity gap, i.e., a *flowlet*. This abstraction better captures the communication

units relevant to transport performance, for which FCT is our primary metric.

Flow-Aware Transport in DCNs. Short and long flows impose different performance requirements, motivating congestion-control mechanisms that differentiate traffic according to flow size [2]–[4]. The key challenge is obtaining this information sufficiently early. Fork [3] assumes exact flow size before transmission, providing an oracle-like upper bound but requiring application-provided information. Reactive approaches instead infer flow behavior after transmission starts: flow-aware DCTCP [2] reclassifies flows after a byte threshold is exceeded, while TCP send-buffer occupancy provides another coarse signal [8], [15]. Such approaches delay flow awareness and motivate prediction before or near flow start.

Traffic and Flow-Size Prediction. Network traffic prediction has been extensively studied using statistical [16] and deep-learning methods [17], [18], typically to forecast aggregate link, path, or network demand. Our focus is instead the size of individual flows/flowlets, which can directly inform flow-aware transport decisions.

At this granularity, prior work differs importantly in *when* information about the target flow becomes available. Elephant-flow classification can be performed from the first packet [13]; historical behavior can be used to predict newly arriving flows once their first packet is observed [11]; and two-stage approaches combine first-packet classification with prediction from the first n packets [12]. Other ML approaches classify mice and elephant flows for routing purposes [19]. These methods provide early information, but the target flow has already arrived and contributes packet-level information to the decision.

Our work builds more directly on approaches that predict flow size from preceding communication history and end-host telemetry [9], [10], using models such as LSTMs, gradient-boosted trees, and feed-forward neural networks. We follow this *pre-arrival* prediction paradigm: the next flow is predicted before observing any packet belonging to it. However, existing approaches in this line train predictors for individual workloads. We extend this setting to a *single cross-workload predictor* and additionally evaluate how prediction errors propagate to transport-level FCT.

AI Workloads and Feature Heterogeneity. DCN traffic is highly heterogeneous: representative data-mining workloads contain more than 80% of flows below 10 KB, while large flows carry most of the traffic volume [1]. Yet many public traces lack the fine-grained host/application context needed for pre-arrival flow-size prediction. For example, Facebook’s packet-sampled traces [6] characterize production traffic but do not expose the required per-flow and host-side telemetry.

The Flux dataset [9] provides such instrumentation for distributed workloads including Spark PageRank, K-Means, SGD, and TensorFlow. However, these workloads expose partially different telemetry spaces and substantially different flow-size distributions. Figure 1, for example, contrasts Spark PageRank, where most flows are below 100 KB (Figure 1a),

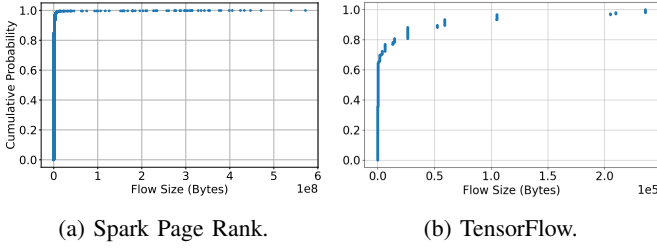


Fig. 1: Flow size CDF for distributed AI workloads.

with TensorFlow, where most are below 10 KB (Figure 1b). Such feature and traffic heterogeneity makes straightforward joint training difficult and motivates our feature-alignment problem.

Position of This Work. Our work combines three aspects. First, we perform *pre-arrival next-flow forecasting*: no packet from the target flow is observed before prediction. Second, we extend workload-specific historical-flow prediction [9], [10] to a single supervised model across heterogeneous AI workloads with partially overlapping telemetry. Third, we evaluate predictions from a transport perspective, quantifying how directional classification errors affect FCT when replacing oracle flow-size knowledge in a flow-aware transport mechanism. The contribution is therefore the combination of *pre-arrival prediction*, *heterogeneous cross-workload learning*, and *transport-level validation*.

III. PREDICTING FLOW SIZE FROM MEASUREMENTS

We formulate flow-size prediction as a supervised regression problem over high-dimensional network telemetry, where the model estimates the total volume y_i of a flow f_i prior to its transmission. The model incorporates a temporal context window that captures the current network state and the feature vectors of the K preceding flows.

Flow Characterization. A flow $f_i \in \mathcal{F}$ is defined as a 6-tuple $\{IP_{src}, IP_{dst}, Port_{src}, Port_{dst}, Proto, Gap_{id}\}$, extending the classical 5-tuple with an additional identifier Gap_{id} . The Gap_{id} distinguishes flows that share the same 5-tuple but are separated in time, i.e., belong to different bursts according to an inter-arrival time criterion. For the purpose of prediction, we define f_i as a sequence of T packets $P_i = \{p_{i,1}, p_{i,2}, \dots, p_{i,T}\}$, where each packet $p_{i,j}$ carries a payload of size $s_{i,j}$.

The total flow size y_i is defined as the sum of the sizes of all packets belonging to the flow until its finalization, where a flow is considered to end when the inter-arrival time between consecutive packets with the same 5-tuple exceeds the time gap Gap_{id} : $y_i = \sum_{j=1}^T s_{i,j}$.

Feature Space and Temporal Windowing. Let each flow f_j be represented by a primary feature vector $\mathbf{z}_j \in \mathbb{R}^d$. This vector includes host-level metrics, network telemetry, and, for completed flows, the realized flow size y_j . We define the components as follows:

- **Target Flow:** f_i is the flow about to start, represented by its initial feature vector $\mathbf{x}_i$, obtained from $\mathbf{z}_i$ by removing the

target variable y_i , i.e., $\mathbf{x}_i \in \mathbb{R}^{d-1} = \mathbf{z}_i \setminus \{y_i\}$. Note that y_i is unknown at prediction time.

- **Historical Context:** $\mathcal{H}_i = \{\mathbf{z}_{i-1}, \mathbf{z}_{i-2}, \dots, \mathbf{z}_{i-K}\}$ represents the sequence of the K most recent flows. Each $\mathbf{z}_{i-k} \in \mathbb{R}^d$ includes its respective ground-truth size y_{i-k} .

The total input representation for the model, referred to as $\mathbf{X}_i$, is obtained by concatenating the current flow features with the flattened features of the K previous flows:

$$\mathbf{X}_i = \mathbf{x}_i \parallel \text{vec}(\mathcal{H}_i) \quad (1)$$

where $\mathbf{X}_i \in \mathbb{R}^{(d-1)+(K \times d)}$. This high-dimensional vector captures the short-term temporal dependencies and workload patterns typical of data center traffic.

In Practice, the feature space can combine application-, host-, and network-level signals, as presented in Section III.

Gradient Boosting Regression. We employ eXtreme Gradient Boosting (XGBoost) [20] to approximate the mapping $h : \mathbf{X}_i \rightarrow \hat{y}_i$. The model is an ensemble of N additive functions:

$$\hat{y}_i = \sum_{t=1}^N f_t(\mathbf{X}_i), \quad f_t \in \mathcal{N} \quad (2)$$

where $\mathcal{N}$ is the space of regression trees. The learning process minimizes a regularized objective function that combines a loss metric L and a penalty for model complexity Ω :

$$\mathcal{L} = \sum_i L(y_i, \hat{y}_i) + \sum_t \Omega(f_t) \quad (3)$$

Given the heavy-tailed nature of flow size distributions, we adopt the Mean Squared Logarithmic Error (MSLE) as the training objective to ensure robustness across multiple orders of magnitude.

$$L(y, \hat{y}) = \frac{1}{2} (\log(y+1) - \log(\hat{y}+1))^2$$

Information Flow and Prediction. By incorporating $\mathcal{H}_i$, the XGBoost model can perform conditional branching based on the performance of past flows. For instance, if the previous K flows from a specific host exhibited high volume and high inter-arrival times, the ensemble of trees can identify this pattern as a specific workload signature and adjust the prediction $\hat{y}_i$ accordingly before the first packet of f_i is even processed.

Why Gradient-Boosted Decision Trees? Following previous work [9], we evaluate three model families for flow-size prediction: feed-forward neural networks (FFNNs), recurrent neural networks with LSTM layers, and GBDTs.

Because our target use case is integration with the transport layer, the predictor must operate with low latency and computational overhead. To effectively reduce FCT, the cost of generating predictions should not outweigh the performance gains they enable [21]. **Figure 2 compares inference latency under identical hardware conditions on a standard server. These measurements are intended for comparative analysis rather than as representative of an optimized deployment; nevertheless, XGBoost achieves substantially lower inference**

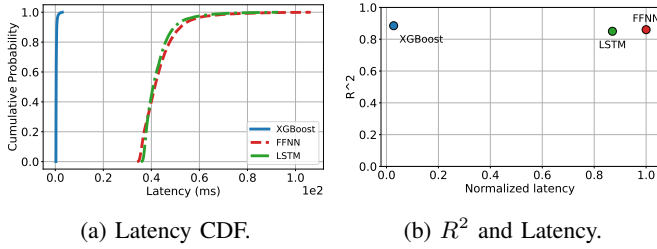


Fig. 2: Inference latency at the 99th percentile.

latency than the evaluated FFNN and LSTM models at comparable predictive performance.

For an XGBoost model with T trees of maximum depth D , inference requires traversing at most D nodes per tree, yielding a per-flow prediction complexity of $\mathcal{O}(TD)$. Constructing the temporal input from the preceding K flows and F features requires $\mathcal{O}(KF)$ operations; since K is fixed and small in our setting, this overhead is bounded and independent of the total number of previously observed flows. The overall online complexity per prediction is therefore $\mathcal{O}(KF + TD)$, with state requirements of $\mathcal{O}(KF)$ for maintaining the historical context. Thus, inference cost does not grow with the total trace length or number of active flows.

Practical deployment additionally requires collecting the end-host telemetry and maintaining the K -flow history used by the predictor; these costs depend on the instrumentation and flow-arrival rate and are not captured by inference latency alone. Tree-based models nevertheless offer favorable deployment properties: they can be implemented in the kernel with lower latency [22] or offloaded to the data plane [23], [24]. We therefore view the reported latency and complexity results as evidence of computational feasibility, while a full transport-stack implementation and measurement of end-to-end telemetry and integration overhead remain future work.

Cross-Workload Model Construction

A cross-workload model provides a single, robust inference engine capable of simultaneously identifying and adapting to diverse traffic signatures. By training on a heterogeneous dataset comprising multiple workload types, the model learns a broader representation of network behavior. This *global* view allows the system to capture shared underlying patterns, such as the relationship between host CPU state and flow burstiness, which are often ignored by models focused on a single application.

A key challenge in combining datasets from different workloads is the heterogeneity and discrepancy of their feature spaces. Differences in monitoring tools and workload-specific instrumentation lead to partially overlapping feature sets, which result in a sparse representation when datasets are merged. To address this issue, we evaluate different strategies, including (i) *Intersection*: restricting the feature space to those features common to all workloads. While simple, this approach discards potentially informative, workload-specific features; (ii) *Stacking and Ensembles*: training separate base models

per workload and combining their outputs through a meta-regressor. This preserves specialization but increases system complexity; (iii) *Union Strategy*: we define a global feature space $\mathcal{X}_{union}$ as the union of all features across datasets, thereby retaining all available information.

The intersection approach is the simplest to implement but suffers from significant information loss, as it forces all workloads into a “lowest common denominator” feature set, often stripping away the high-value, workload-specific metrics that drive accuracy. Stacking and Ensembles should achieve acceptable precision by preserving specialized base learners, but they introduce operational complexity; adding a new workload requires retraining multiple models and a new meta-regressor, making the system difficult to scale. The Union Strategy provides a practical balance. While it creates a higher-dimensional and sparse feature space, it preserves all available information across datasets. The primary advantage is its modularity: when a new workload is introduced, the system simply appends new feature columns. This balance, along with the accuracy limitations observed in models built with the former approaches, is our main motivation for focusing on the Union approach.

The Union Strategy. To extend the formalization to a cross-workload setting, we must address the structural divergence between datasets from diverse applications (e.g., KMeans vs. TensorFlow). We define a global feature space $\mathcal{X}_{union} = \bigcup_{w \in \mathcal{W}} \mathcal{X}_w$, where $\mathcal{W}$ is the set of all considered workloads.

For a flow f_i belonging to workload $w \in \mathcal{W}$, the unified feature vector $\mathbf{X}_i^{union}$ is constructed as follows:

$$\mathbf{X}_i^{union} = \begin{cases} x_{i,j} & \text{if feature}_j \in \mathcal{X}_w \\ \text{NaN} & \text{if feature}_j \notin \mathcal{X}_w \end{cases} \quad (4)$$

By concatenating this unified vector with the historical window defined before, the input for the model becomes:

$$\mathbf{V}_i = \mathbf{X}_i^{union} \parallel \text{vec}(\mathcal{H}_i^{union}) \quad (5)$$

This formalization relies on XGBoost’s sparsity-aware algorithm. Unlike standard regression techniques that require imputation (e.g., mean substitution), the GBDT objective natively handles NaN values by treating them as a separate category. During training, for each split at node v , the model learns a default direction $D_v \in \{(L)eft, (R)ight\}$ for missing values that minimizes the training loss:

$$\mathcal{L} = \sum_{i \in I_L \cup \{i \in I_{missing} : D_v = L\}} L_i + \sum_{i \in I_R \cup \{i \in I_{missing} : D_v = R\}} L_i \quad (6)$$

where $I_{missing}$ denotes the set of samples with missing feature values. This allows a single global model to act as a specialized predictor for each specific workload w when relevant features are present, while maintaining a baseline prediction when they are absent.

Classification. The same cross-workload strategy can also be applied to flow-size classification. Rather than estimating the continuous size y_i , the model directly predicts whether a

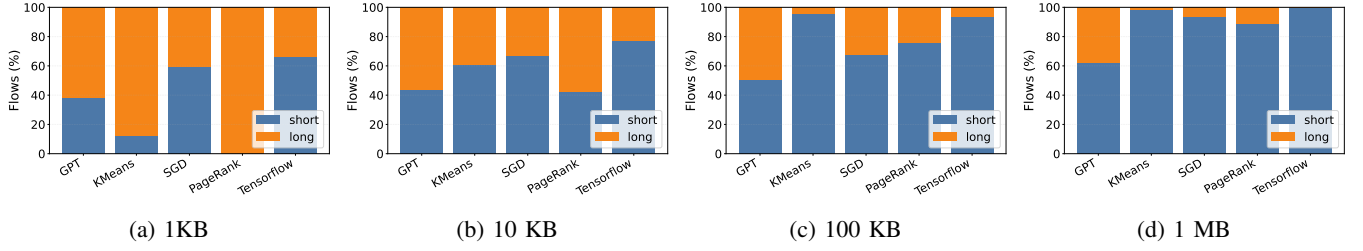


Fig. 3: Distribution of short and long flows across AI workloads for four representative short-flow thresholds.

TABLE I: Feature groups considered by the models.

Feature Group	Examples
Temporal context	time, previous_flow_time, flow_frequency
Recent flow history	flow_size1..K, previous_flow_time1..K
Host activity	cpu, ram, machine, first_call
Memory and storage	memory_read, memory_write, disk_read, disk_write
Network activity	agg_net_in, agg_net_out, per-machine counters
Transport state	tcp_queue, send/receive-related counters
Feature alignment	workload identifier, missing-feature indicators

flow falls below a given size threshold, which is the decision required by transport mechanisms that differentiate short and long flows. For this, we also formulate flow-size awareness as a binary classification problem. Given a threshold θ that separates short and long flows, each flow f_i is assigned a class label

$$c_i = \begin{cases} 1, & \text{if } y_i > \theta, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

where $c_i = 1$ denotes a long flow and $c_i = 0$ denotes a short flow. The classifier estimates the probability that a flow belongs to the long-flow class:

$$\hat{p}_i = g(\mathbf{V}_i), \quad (8)$$

where V_i is the union-based representation defined in Equation 5. The predicted class is obtained as

$$\hat{c}_i = \begin{cases} 1, & \text{if } \hat{p}_i \geq \tau, \\ 0, & \text{otherwise,} \end{cases} \quad (9)$$

where τ is the decision threshold. **For each model, τ is selected on the validation set as the value that maximizes the F1 score.**

We train the classifier using XGBoost with a binary logistic objective. The optimization minimizes the regularized binary cross-entropy loss:

$$\mathcal{L}_{\text{cls}} = - \sum_{i=1}^n [c_i \log(\hat{p}_i) + (1 - c_i) \log(1 - \hat{p}_i)] + \sum_{t=1}^N \Omega(f_t).$$

As in the regression case, the classifier is trained over the global union feature space $\mathcal{X}_{\text{union}}$ and relies on XGBoost’s sparsity-aware handling of missing values. This allows a single classifier to exploit workload-specific features when they are available, while still producing predictions for workloads where those features are absent.

System Design. The predictor relies on telemetry collected at the end hosts, where application activity, system behavior, and

communication events can be observed close to the source of traffic generation. This host-side perspective is important because flow size is not only determined by the network, but also by computation, memory, and storage activity, and previous communication patterns. Accordingly, Table I summarizes the feature groups considered in this work. The feature selection aims to preserve predictive information while aligning heterogeneous telemetry spaces. To capture temporal context, each prediction uses the current flow and the previous K flows. The parameter K controls how much recent communication history is exposed to the model. **We treat K as a hyperparameter and evaluate multiple history lengths on the validation set. We select $K = 5$, which maximizes validation R^2 ; this value also coincides with that reported in prior work [9].**

The target variable, `flow_size0`, is excluded from the input. To prevent leakage, all features correspond to observations available before the target flow starts, and historical features use only preceding flows.

Our data combines public and newly collected datasets. We use the Flux dataset [9] as a public baseline because it provides flow-level and system-level instrumentation for distributed AI workloads using CPUs and GPUs at scale. However, Flux reflects an older software and workload environment. Consequently, we extend this dataset with additional traces collected using INTFusion [25]. These traces include an LLM fine-tuning workload based on GPT, as well as updated Spark workloads for KMeans, PageRank, and SGD collected with newer versions of the framework. This combination allows us to compare against a public reference dataset while assessing whether cross-workload prediction remains effective on updated and more representative AI data center workloads. **Model Training.** Although the primary prediction task is formulated as regression, to evaluate classification, we used common short flow thresholds of 1 KB, 10 KB, and 100 KB. To further characterize the data, Figure 3 illustrates the class distribution across the evaluated datasets. As observed, the balance is different among the workloads and thresholds.

IV. EVALUATION

The evaluation is guided by the following research questions: **RQ1:** *How well does a unified cross-workload model predict flow size compared with workload-specific predictors across heterogeneous AI workloads?* **RQ2:** *How accurately does the unified model classify short and long flows for flow-aware transport?* **RQ3:** *How well does the unified model*

TABLE II: Simulation setup.

Parameter	Values
Link Speeds	10 Gbps
RTT	60 μ s
Buffers	250 KB
Loads	30–90%
Short-flow fraction	80%
Short-flow sizes	1–100 KB
Long-flow sizes	1–10 MB
Flow-size distribution	Empirical DCN distribution
Flow arrival process	Poisson
Short-flow threshold	100 KB

generalize to workload types entirely unseen during training?

RQ4: How does replacing Fork’s oracle flow-size knowledge with learned predictions affect FCT?

A. Experimental Methodology & Datasets

We evaluate the proposed predictors from both machine-learning and networking perspectives. At the prediction layer, we assess regression performance using R^2 , MAE, and RMSE, and compare the proposed cross-workload predictor against workload-specific models to quantify the cost of replacing per-workload specialization with a unified model. We further evaluate classification performance using accuracy, precision, recall, and F1 score. At the transport layer, we assess the impact of prediction errors on congestion-control decisions by injecting the learned predictions into Fork in ns-3 using the configuration in Table II. Each transport configuration is repeated across independent random seeds, and we report mean results with confidence intervals. Fork provides a strong baseline because it assumes perfect short/long flow identification and has demonstrated improved performance over state-of-the-art DCN transports such as Homa [26].

We use the open dataset introduced in [9], comprising four large-scale distributed AI workloads, and complement it with unseen workloads collected on a small university cluster using INTFusion [25], a telemetry architecture for generating new workload traces [14]. We evaluated several XGBoost-based configurations, including feature-intersection and stacking variants; since these consistently yielded lower predictive quality, the evaluation focuses on workload-specific predictors and the proposed cross-workload predictor.

The dataset comprises multiple tabular files per workload, each corresponding to an independent workload run. We split the data at the file level: every run is assigned entirely to the training, validation, or test set using a 70/20/10 split, preventing flows from the same run from appearing across partitions. Prediction is performed on normalized flow-size values rather than raw byte values: each feature column, including the target, is scaled by the maximum value computed from the training data. Raw flow-size values are reconstructed only after prediction. **Max normalization preserves the non-negative and sparse structure of the telemetry features and provides a simple inverse transformation. We note that XGBoost does not inherently require feature normalization; the transformation is primarily used to maintain a consistent representation across heterogeneous workload traces.**

We distinguish between two levels of generalization. In the standard cross-workload evaluation, the training, validation, and test partitions contain independent runs of the same set of workload types. The file-level split described above prevents leakage across executions and therefore evaluates generalization to unseen runs of workloads represented during training. We refer to this setting as *cross-run generalization*.

To evaluate generalization to an entirely unseen workload, we additionally perform a leave-one-workload-out (LOWO) evaluation. For each workload w , all of its runs are held out, while the model is trained and validated exclusively on the remaining workloads. After model selection and threshold calibration, both the model and classification threshold are fixed and evaluated on w . Thus, no samples from the held-out workload are used during training, validation, or threshold selection, providing a zero-shot evaluation on an unseen workload type.

B. RQ1: Cross-Workload Prediction Performance

To quantify the predictive cost of replacing workload-specific models with a cross-workload predictor, Table III compares both approaches using R^2 , MAE, and RMSE. R^2 captures the fraction of per-workload variance explained by the model, MAE quantifies the typical prediction error, and RMSE places greater weight on large residuals. We additionally report relative differences between the cross-workload and workload-specific models to facilitate comparison across workloads.

The cross-workload regressor remains competitive across heterogeneous workloads, although the effect of unification is workload-dependent. For GPT, the unified model slightly improves all three metrics, and for KMeans it reduces both MAE and RMSE while preserving a comparable R^2 . These results indicate that joint training can transfer useful predictive structure across some workloads. In contrast, SGD, PageRank, and TensorFlow exhibit higher absolute errors under the cross-workload model. The largest reduction in R^2 occurs for SGD, suggesting that part of its communication behavior remains workload-specific and is not fully preserved by feature alignment and joint training. TensorFlow remains highly predictable under both configurations, although the workload-specific model achieves lower MAE and RMSE.

The gap between MAE and RMSE indicates that a relatively small number of large residuals contributes disproportionately to squared-error-based metrics. This behavior is consistent with the heavy-tailed nature of flow-size distributions and motivates the evaluation of prediction quality beyond aggregate regression metrics. Figure 4 therefore reports the threshold-crossing error induced when continuous regression outputs are converted into short/long flow decisions. A threshold-crossing error occurs when the predicted and actual flow sizes lie on opposite sides of the selected boundary. Similar regression performance does not necessarily translate into similar transport behavior, since errors near the short/long decision threshold can change the flow class assigned by the transport protocol, whereas even larger errors away from the threshold may have no impact on transport.

TABLE III: Regression performance of workload-specific and union models. Positive improvement indicates better performance of the union model (higher R^2 or lower prediction error).

Workload	R^2			MAE (KB)			RMSE (KB)		
	Specialized	Union	Imp. (%)	Specialized	Union	Imp. (%)	Specialized	Union	Imp. (%)
GPT	0.16	0.18	12.5	1322	1311	0.8	9428	9334	1.0
KMeans	0.88	0.87	-1.1	669	562	16.0	7739	7083	8.5
SGD	0.63	0.54	-14.3	37	62	-67.3	316	350	-10.8
PageRank	0.87	0.85	-2.3	467	524	-12.2	4940	5284	-7.0
TensorFlow	0.97	0.94	-3.1	1	3	-203.5	8	11	-41.1

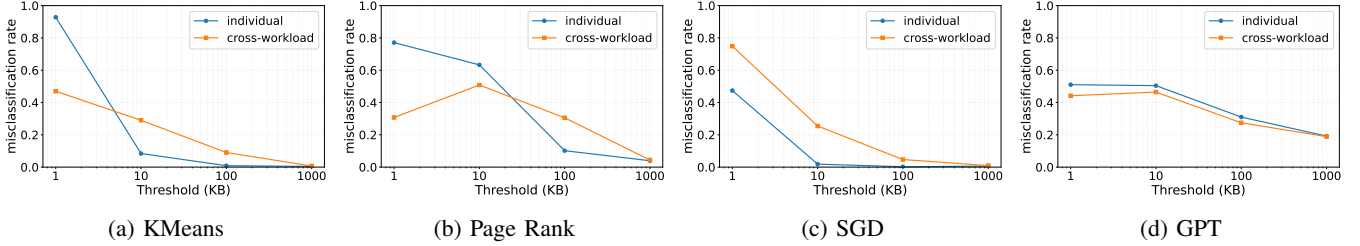


Fig. 4: Misclassification rate per short-flow threshold.

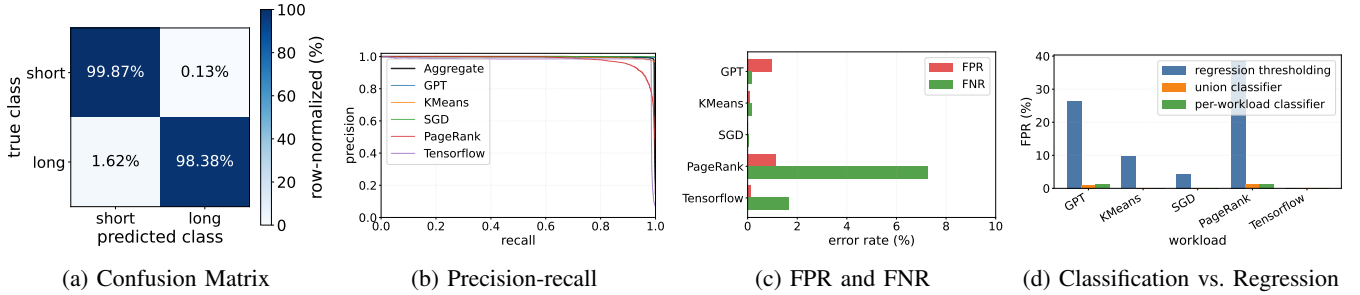


Fig. 5: Summary of the classification performance metrics.

Take-away RQ1

A single cross-workload predictor can support heterogeneous AI workloads through a common model, at the cost of workload- and metric-dependent degradation relative to specialized predictors. While larger gaps arise for some regression metrics, the unified approach provides a practical trade-off between predictive performance and the complexity of maintaining separate workload-specific models.

C. RQ2: Cross-Workload Classification

Although regression estimates continuous flow sizes, flow-aware transport ultimately requires classifying flows as short or long. Consequently, the most relevant prediction errors are those that cross the decision threshold rather than the largest absolute residuals. We therefore evaluate classification performance using a 100 KB threshold, which provides a representative separation between latency-sensitive short flows and larger transfers while matching the Fork configuration used later.

Figure 5 summarizes the classification performance. The confusion matrix in Figure 5a shows that the cross-workload classifier correctly identifies 99.87% of short flows and

98.38% of long flows. Accordingly, only 0.13% of short flows are incorrectly assigned to the long-flow class, while 1.62% of long flows are classified as short. These two error types are particularly relevant for flow-aware transport: false-long decisions may prevent latency-sensitive short flows from receiving preferential treatment, whereas false-short decisions may place large transfers in the short-flow control loop and increase contention for latency-sensitive traffic.

The precision-recall curves in Figure 5b remain close to the upper-right corner for the aggregate test set and most individual workloads, indicating high precision over a broad recall range. PageRank exhibits the largest precision-recall trade-off and the highest false-negative rate, suggesting that its short and long flows are more difficult to separate around the threshold. TensorFlow also shows a non-negligible false-negative rate, whereas KMeans and SGD exhibit near-zero error rates. GPT has the highest false-positive rate, although it remains below 1%.

Figure 5d further compares direct classification with regression thresholding. The cross-workload classifier substantially reduces the false-positive rate relative to converting regression outputs into short/long labels, particularly for GPT and PageRank. This result highlights that minimizing continuous flow-

TABLE IV: Leave-one-workload-out performance.

Excluded	Accuracy	F1	ROC-AUC	Short→Long	Long→Short
GPT	0.779	0.792	0.716	18.3%	25.0%
KMeans	0.972	0.641	0.981	1.8%	30.7%
PageRank	0.861	0.518	0.746	6.8%	52.4%
SGD	0.923	0.836	0.969	6.9%	10.6%
TensorFlow	0.934	0.025	0.985	0.09%	98.7%

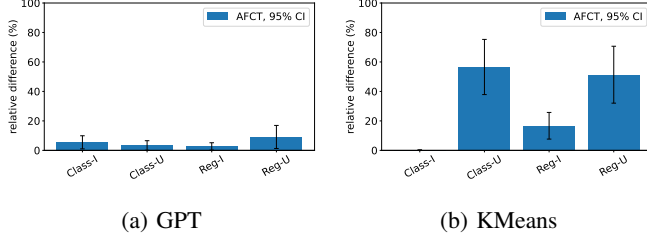


Fig. 6: AFCT relative difference using real traces in Fork.

size error and minimizing threshold-crossing error are related but distinct objectives. Regression remains useful because it supports arbitrary thresholds and richer transport decisions; however, a classifier trained specifically for the 100 KB boundary can more effectively protect short flows from being incorrectly assigned to the long-flow class.

Take-away RQ2

Despite the limitations of continuous flow-size regression, a unified cross-workload classifier can reliably distinguish short from long flows, achieving over 98% classification accuracy across heterogeneous AI workloads. For flow-aware transport, the most relevant errors are those that cross the short/long decision boundary, particularly underestimating long flows, which disproportionately impact transport performance.

D. RQ3: Generalization to Unseen Workloads

We now consider the zero-shot setting in which the workload type itself is entirely unseen during training. For each LOWO experiment, one workload is excluded from training and validation, and the resulting model is evaluated exclusively on that workload. We use the 100 KB short-flow threshold to maintain consistency with the classification and Fork evaluations presented next.

Table IV shows that zero-shot transfer is feasible, but strongly workload dependent. The held-out SGD workload exhibits the strongest overall transfer, achieving an F1 score of 0.836 and ROC-AUC of 0.969, while limiting the short-to-long and long-to-short error rates to 6.9% and 10.6%, respectively. KMeans also retains excellent ranking capability, reaching a ROC-AUC of 0.981 and an overall accuracy of 0.972. However, 30.7% of its long flows are still classified as short, showing that strong aggregate performance can conceal class-specific errors. PageRank is substantially more challenging in the zero-shot setting, and GPT exhibits a more balanced degradation, with short-to-long and long-to-short error rates of 18.3% and 25.0%, respectively. TensorFlow provides the clearest example of the distinction between ranking performance

and the fixed classification operating point. Despite achieving a ROC-AUC of 0.985 and an overall accuracy of 0.934, its F1 score drops to 0.025 because 98.7% of long flows are classified as short, while only 0.09% of short flows are classified as long. Thus, the model preserves a strong ability to rank TensorFlow flows by their likelihood of belonging to the long-flow class, but the decision threshold learned without TensorFlow does not transfer effectively to this unseen workload.

These results characterize how the proposed framework extends beyond its primary supervised cross-workload setting to entirely unseen workloads. Zero-shot transfer is clearly workload-dependent: while some workloads generalize well, others exhibit substantial class-specific errors and would require retraining or adaptation. The results also highlight the importance of complementing aggregate ML metrics with class-specific and transport-oriented evaluation, since high accuracy or ROC-AUC can mask long-to-short misclassifications that our transport analysis shows to be particularly consequential for FCT.

Take-away RQ3

Zero-shot generalization to entirely unseen workload types is possible but strongly workload dependent. While SGD retains strong and balanced performance, PageRank and TensorFlow exhibit substantial long-to-short errors, indicating that some unseen workloads require retraining or adaptation. TensorFlow further shows that high ROC-AUC does not necessarily translate into an effective classification operating point, highlighting the importance of class-specific and transport-oriented evaluation when onboarding new workloads.

E. RQ4: Transport-Layer Validation

Finally, we evaluate whether learned flow-size prediction can effectively replace Fork’s oracle flow-size knowledge in a realistic flow-aware transport protocol. To this end, we integrate the workload-specific and cross-workload models (m) into Fork running in ns-3 under trace-driven traffic and quantify their impact using the relative average flow completion time (AFCT) difference with respect to the original Fork configuration, which assumes perfect short/long flow identification:

$$\Delta_{\text{AFCT}}(m) = \frac{\text{AFCT}_m - \text{AFCT}_{\text{Fork}}}{\text{AFCT}_{\text{Fork}}} \times 100.$$

A value of 0% indicates performance identical to the Fork-oracle baseline, whereas positive values represent the AFCT penalty incurred by replacing perfect flow-size knowledge with learned predictions. This normalization enables meaningful comparison across workloads with different traffic characteristics and baseline FCTs.

Figure 6 reports Δ_{AFCT} for four prediction settings: workload-specific and cross-workload classifiers (Class-I and Class-U), and workload-specific and cross-workload regressors (Reg-I and Reg-U). Real traces from the test partition are

replayed as foreground traffic, while the statistical workload of Table II provides background traffic.

The transport impact of model generalization is workload-dependent. For KMeans, the workload-specific classifier remains closest to the Fork-oracle baseline, whereas the cross-workload classifier and regression-based variants incur larger AFCT penalties. In contrast, GPT exhibits only minor differences across all prediction approaches, with both classifiers remaining close to the oracle configuration and the regression models introducing only limited degradation. These results are consistent with the prediction-layer analysis: cross-workload learning preserves transport performance when workload patterns transfer well, whereas degradation becomes visible when threshold-crossing errors alter Fork’s short/long flow assignment. More generally, transport behavior cannot be inferred from aggregate prediction accuracy alone. Predictors with similar regression or classification metrics may produce different AFCTs if their remaining errors occur near the decision threshold or systematically classify long flows as short. Conversely, modest reductions in predictive accuracy have little impact on transport when most prediction errors remain on the correct side of the threshold.

Finally, to better understand the transport results, we independently vary short- and long-flow classification accuracy while measuring the resulting AFCT. Figure 7 reports the relative AFCT penalty with respect to the Fork-oracle baseline, where the horizontal and vertical axes represent long- and short-flow prediction accuracy, respectively.

The results reveal a strong asymmetry between the two error types. Long-flow accuracy below approximately 90–95% frequently more than doubles AFCT, regardless of short-flow accuracy. In contrast, once long-flow accuracy reaches 98–99%, AFCT remains close to the oracle baseline over a broad range of short-flow accuracies. This behavior reflects Fork’s higher sensitivity to underestimating large flows, which incorrectly routes them through the short-flow control loop and increases contention for latency-sensitive traffic.

Conversely, overestimating short flows has a smaller and less systematic impact, as it only prevents preferential treatment without introducing large transfers into the short-flow path. These results explain the previously observed workload-dependent AFCT and identify long-to-short misclassification as the dominant source of transport degradation.

Take-away RQ4

Integrating the proposed predictors into Fork shows that learned flow-size prediction can effectively replace oracle flow-size knowledge with only limited transport degradation. The unified cross-workload predictor preserves most of the transport benefits of workload-specific models while eliminating the need for one predictor per workload. Transport performance is primarily determined by long-to-short misclassification, highlighting the importance of minimizing threshold-crossing errors.

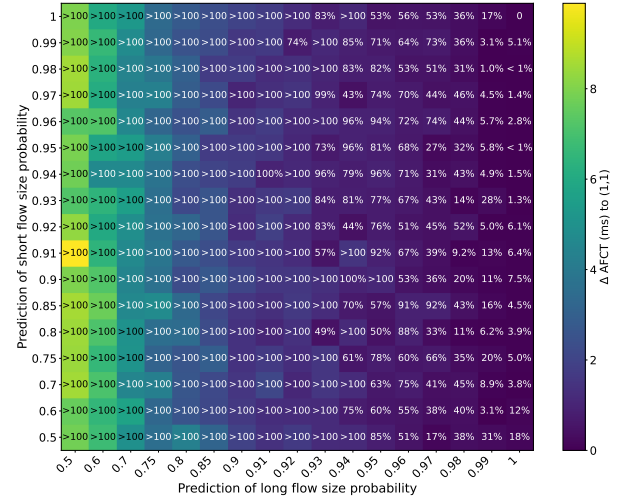


Fig. 7: Short and long flow sensitivity analysis over an oracle.

V. CONCLUSION

This paper presents a unified cross-workload flow-size prediction framework for AI data center networks that aligns heterogeneous end-host telemetry and eliminates the need for workload-specific predictors. Our results show that a single model preserves most of the predictive quality and transport benefits of specialized models, establishing cross-workload flow-size prediction as a practical foundation for learning-assisted congestion control without requiring oracle flow-size knowledge or one model per workload. More importantly, we show that transport-aware evaluation is essential: similar regression performance can lead to markedly different transport behavior, whereas minimizing threshold-crossing errors and preserving long-flow identification translate more directly into improved flow completion times. We hope these findings encourage closer co-design of machine learning models and transport protocols for future AI data center networks.

REFERENCES

- [1] M. Noormohammadpour et al., “Datacenter Traffic Control: Understanding Techniques and Tradeoffs,” *IEEE Comm. S. & T.*, vol. 20, pp. 1492–1525, 2018.
- [2] S. Joy and A. Nayak, “Improving flow completion time for short flows in datacenter networks,” in *IFIP/IEEE IM*.
- [3] Y. Liu et al., “Fork: A Dual Congestion Control Loop for Small and Large Flows in Datacenters,” in *EuroSys*, 2025.
- [4] L. Alberro and E. Grampin, “Transport protocols in the data center: Understanding proposals and research challenges,” *Computer Networks*, vol. 274, p. 111793, 2026.
- [5] M. Li et al., “ScaleAcross Explorer: Exploring Communication Optimization for Scale-Across AI Model Training,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.24326>
- [6] A. Roy et al., “Inside the Social Network’s (Datacenter) Network,” in *SIGCOMM’15*, 2015.
- [7] T. Benson et al., “Understanding data center traffic characteristics,” *SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 1, p. 92–99, Jan. 2010.
- [8] A. R. Curtis et al., “Mahout: Low-overhead datacenter traffic management using end-host-based elephant detection,” in *INFOCOM*, 2011.
- [9] V. Dukić et al., “Is advance knowledge of flow sizes a plausible assumption?” in *NSDI*, 2019.
- [10] S. M. Hosseini et al., “Flow Size Prediction with Short Time Gaps,” in *IEEE INFOCOM Workshops*, 2024.
- [11] M. V. B. da Silva, A. S. Jacobs, R. J. Pfitscher, and L. Z. Granville, “Predicting Elephant Flows in Internet Exchange Point Programmable Networks,” in *Proc. Int. Conf. Advanced Information Networking and Applications (AINA)*, 2019.
- [12] S. Xie, G. Hu, C. Xing, and Y. Liu, “Online Elephant Flow Prediction for Load Balancing in Programmable Switch-Based DCN,” *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 1, pp. 745–758, 2024.

- [13] P. Jurkiewicz *et al.*, “Machine Learning-Based Elephant Flow Classification on the First Packet,” *IEEE Access*, vol. 12, 2024.
- [14] L. Alberro *et al.*, “Unified Learning Repository,” <https://github.com/leolb/unified-learning>, 2026, accessed: 2026-08-21.
- [15] G. Wang *et al.*, “c-Through: part-time optics in data centers,” in *SIGCOMM '10*, 2010.
- [16] A. Bayati *et al.*, “Gaussian process regression ensemble model for network traffic prediction,” *IEEE Access*, vol. 8, pp. 176 540–176 554, 2020.
- [17] X. Ma *et al.*, “Cellular network traffic prediction based on correlation convlstm and self-attention network,” *IEEE Comm. Lett.*, vol. 27, pp. 1909–1912, 2023.
- [18] M. Li *et al.*, “A deep learning method based on an attention mechanism for wireless network traffic prediction,” *Ad Hoc Networks*, vol. 107, 2020.
- [19] P. Poupart *et al.*, “Online flow size prediction for improved network routing,” in *IEEE ICNP*, 2016.
- [20] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *ACM KDD*, 2016.
- [21] G. Wan *et al.*, “CATO: End-to-End optimization of ML-Based traffic analysis pipelines,” in *22nd USENIX NSDI Symposium*, pp. 1523–1540, 2025.
- [22] DMLC Contributors, “Treelite: Universal Model Exchange and Serialization Format for Decision Tree Forests,” <https://github.com/dmlc/treelite>, 2026.
- [23] C. Zheng *et al.*, “Planter: Rapid Prototyping of In-Network Machine Learning Inference,” *SIGCOMM Comput. Commun. Rev.*, vol. 54, p. 2–21, 2024.
- [24] B. Brandino *et al.*, “Detecting Attacks at Switching Speed: AI/ML and Active Learning for in-Network Monitoring in Data Planes,” in *ICNP Workshops*, 2024.
- [25] L. Alberro *et al.*, “INTFusion: Unifying Network and Host Telemetry in Data Center Networks,” in *IFIP Networking*, 2026.
- [26] B. Montazeri *et al.*, “Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities,” in *SIGCOMM'18*, 2018.